

AI224: Operating Systems — Lecture 12

Dr. Karim Lounis

Jan - May 2025



Thrashing

Thrashing

Definition

Thrashing. Occurs when a process spends most of its time paging (“page faulting”) than executing due to the fact of having not enough frames.

Example. Imagine a process has 2 frames, 1 for code and 1 for data. At a given point in time, the process requires access to a second data frame to be used with the first data frame at the same time. The process thrashes.

When a process does not have enough frames, it will page-fault and replace a page that will be needed again right away (i.e., will constantly page-faults).

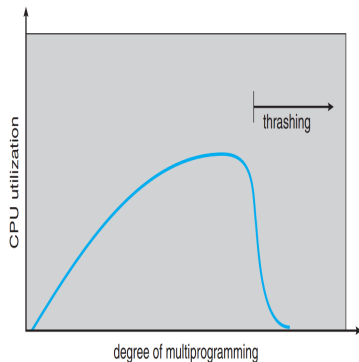
When there is a big # of processes in the memory, certain processes may need more frames & hence start “stealing” frames from other processes.

Now the processes which have been “stolen” the frames, will also require their frames back and hence fault and “steal” frames from others.

Thrashing

The number of page faults will drastically increase, creating a long queue at the HDD, whereas the ready queue is getting emptied.

- If a $\#$ of processes are thrashing, i.e., spending most of their time in the HDD queue, the page fault service time increases (due to long queue).
- In fact, if a thrashing process steals frames from another process, it may cause the latter to thrash as well (global replacement Algo is used)
- This also increases the effective access time of a process that is not thrashing (system bus and memory are busy).



Thrashing Prevention

To limit thrashing we can:

- Use a **local replacement algorithm**, where a process cannot “steal” frames from other processes.
- Provide processes with as many frames as they **need**.

Applying a local page replacement algorithm is fine, however, knowing how many pages are needed by a process is a problem?

To deal with this issue, we can use one of the following algorithms: **Working-Set (WS)** algorithm and the **Page Fault Frequency (PFF)** algorithm.

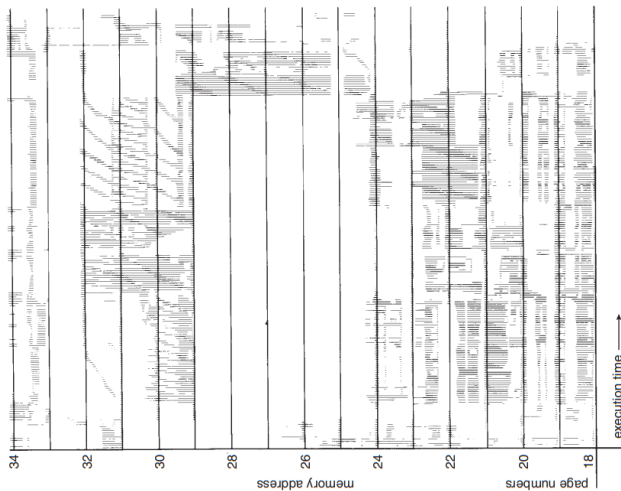
Definition

Locality. Is a set of pages that are actively used together at a specific instant of time during a process execution.

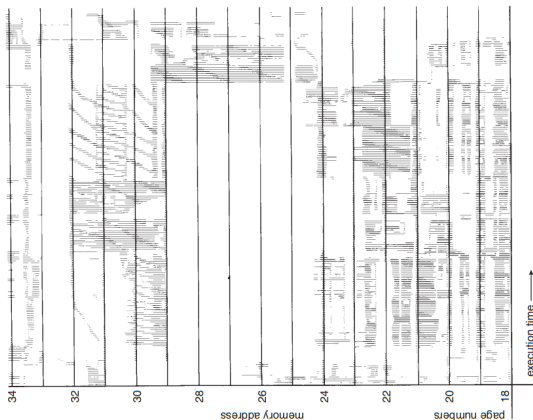
A program is composed of several different localities, which may overlap.

As a process executes over time, it moves from locality to locality.

Thrashing Prevention



Thrashing Prevention



Based on this figure, what can we provide processes with to prevent them from thrashing?

Thrashing Prevention

Allocating enough frames for each locality:

- By allocating enough frames to a process to accommodate its current locality, the process will only fault for the pages in its locality until all these pages are in memory (i.e., paged in).
- The program will not fault **again** till it changes the locality.

If we do not allocate enough frames for each locality:

- The program will **thrash** since it cannot keep all the page that composes its current locality in the memory.

Hence, to limit thrashing we can:

- Use a **local replacement algorithm**, where a process cannot “steal” frames from other processes.
- Provide processes with enough frames to accommodate its current locality (**working-set**).

Working-Set Model

Definition

Working Set Algo. Estimates the amount of frames needed by a process in its next execution phase based on its current memory requirement to prevent it from thrashing while keeping the degree of multiprogramming as high as possible. It is based on the assumption of locality.

The Algo uses a parameter Δ , which defines the **Working Set Window**.

The idea is to examine the most recent Δ page references made by a given process to build its **WS**, i.e., a set of frame what will host its locality.

Working set. Is the set of pages that are actively in use $\approx$ locality.

Example. Consider the following page references from a process ($\Delta = 5$):

... 1 2 5 6 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 4 3 4 4 1 3 2 3 4 4 3 ...
 $WS(t_1) = \{1, 2, 5, 6, 7\}$ $WS(t_2) = \{2, 3, 4\}$

Working-Set Model

The accuracy of the working set depends on the value of Δ :

If Δ is too small, it will not encompass the entire locality. However, if it is too big $\Delta \rightarrow \infty$ then the working set will contain all the pages that the process used during its execution (so, a trade-off has to be made).

The most important variable in the working set is its size WSS , which depends on Δ (each process has WSS at a given time t)

If we consider a number of processes (say N), then the total demand for frames would be:

$$D = \sum_i^N WSS_i$$

Thus, if the RAM contains m available frames for processes, thrashing will not occur when:

$$D \leq m$$

Otherwise, if $D > m$, then at least one process is thrashing, and a victim process should be swapped out (put to sleep).

Working-Set Model

- The OS monitors the working-set of each process and allocates to that working-set enough frames to provide it with its working-set size.
- If frames are left, another process can be started (i.e., $\sum_i^N WSS_i < m$).
- When the sum of the working-set increases and exceeds the total number of available frames **m**. The OS suspends a **victim** process, **reducing the degree of mutlti-programming**.
- Based on the size of the working-set (**because it is variable**), more frame can be allocated or de-allocated to/from the process.

Example. Games have a constant working set — the amount of memory actively needed to run smoothly. If this exceeds physical RAM, the system starts thrashing (excessive paging), leading to poor performance.

Regarding the parameter Δ :

- **Fixed System-Wide Window.** In some implementations, a single time window size (Δ) might be chosen for all processes in the system.
 - Simple algorithm.
 - Not ideal for all processes ($\neq$ access patterns).
- **Dynamic Window per Process.** Dynamically adjusting the time window size (Δ) for each individual process.
 - Better adaptation to the specific locality of reference of each process.
 - Computationally more expensive.

Demo

Working-Set Model

Exercise. Consider the following page referencing sequences of processes P1 and P2. Calculate the number of allocated frames to each process at time $t=6$ and $t=12$ using the working-set algorithm for thrashing prevention. Also specify for each case whether the OS can initiate a new process P3 that requires only one frame allocation or must terminate a process, considering that the total number of available frames in RAM is 7 frames.

Time	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
P1	4	5	6	4	3	4	3	3	5	3	7	4	7	4	7	6	7	7
P2	2	2	3	2	3	2	3	3	2	2	2	9	5	2	9	5	8	6

① $\Delta=6$ and $t=6$?

② $\Delta=6$ and $t=12$?

Page-Fault Frequency

Another direct approach to prevent thrashing is the Page-Fault Frequency.

It relies on the following two observations:

- 1 When the page-fault rate of a process is very high, this implies the process may be thrashing, and hence needs more frames.
- 2 When the page-fault is too low, this implies that the process may have too many frames.

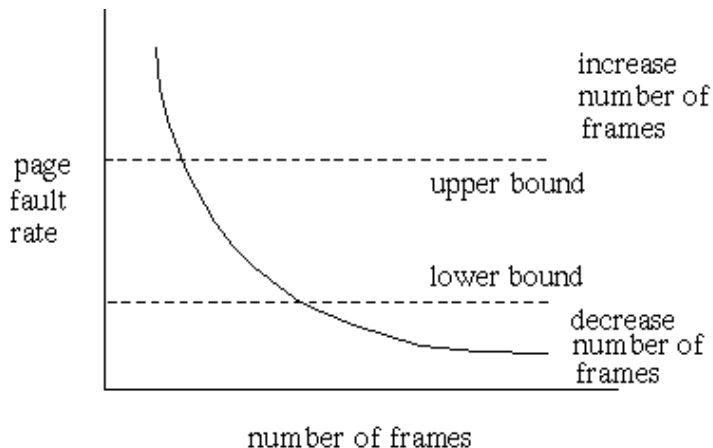
To establish upper and lower bounds on a desired page fault rate τ .

PFF measures the CPU time between two consecutive page faults for each process:

- If the time is short (high rate), give process more frames (if no frame, spool out some processes).
- Otherwise, if the time is long (low rate), take away some frames (those not used since last PF, restart suspended process).

Page-Fault Frequency

Good algorithm since computation occurs at page faults, not at each memory reference.



Memory management

End of Memory Management

Process Synchronization

Monitors for Synchronization

Problems with semaphores

Although semaphores are convenient and effective mechanisms for process synchronization, using them incorrectly can result in **timing errors** that are difficult to detect. For example:

- 1 *A programmer interchanges the order of `acquire()` and `release()`.*

```
{release(s);...; CS ;...; acquire(s);}
```

- 2 *A programmer replaces a `release()` by `acquire()`.*

```
{acquire(s);...; CS ;...; acquire(s);}
```

- 3 *A programmer omits `acquire()` or `release()`.*

```
{acquire(s);...; CS ;...}; or {...; CS ;...; release(s)};
```

The compiler does not detect those bugs.

Problems with semaphores

Although semaphores are convenient and effective mechanisms for process synchronization, using them incorrectly can result in **timing errors** that are difficult to detect. For example:

- 1 *A programmer interchange the order of `acquire()` and `release()`.*

```
{release(s);...; CS ;...; acquire(s);}
```

- 2 *A programmer replace a `release()` by `acquire()`.*

```
{acquire(s);...; CS ;...; acquire(s);}
```

- 3 *A programmer omits `acquire()` or `release()`.*

```
{acquire(s);...; CS ;...}; or {...; CS ;...; release(s)};
```

To deal with such errors, Monitors have been developed.

Monitors

Definition

Monitors (Hoare 74): Are high-level programming language constructs that provides mutual exclusion [and synchronization].

A monitors is an abstract data type that includes:

- A set of programmer-defined operations (procedures or methods) which access is “supposed to be” mutually exclusive.
to implement the entry and exit section or critical section
- A set of synchronization variables called **condition variables**.
to implements waiting queues
- A set of local variables, used to determine the state of the system.
e.g., to keep track of the number of blocked threads

The access to the a monitor is **mutually exclusive**. If a process attempts to execute any of the monitor's functions while another process is currently within the monitor, the former process blocks till the latter finishes.

Monitors

- Operations defined within a monitor can only access those variables declared locally within the monitor as well as their formal parameters.
- If a process is running inside a monitor, it does not imply that the process is executing its critical section.
- A monitor without condition variables is called a **pure monitor**. It only provides mutual exclusion.

Relationship between Monitors and Semaphores:

The compiler will translate monitor source code to a mutually-exclusive accessed area by adding semaphores where needed.

Monitors ain't available in all programming languages but can be simulated:

e.g., in C programming language there is no monitor, but mutex locks (through pthread.h) can be used to simulate monitors.

Other languages support it: Ada, C++, Delphi, Java, Python, Ruby, ...

Monitors

The construct declaring a monitor M with n condition variables, m local variables, and o procedures, has the following form:

Monitor M

```
{  
    Condition variables:   $x_1, x_2, \dots, x_n$ ;  
  
    Local variables:   $v_1, v_2, \dots, v_m$ ;  
  
    void P_1( $p_1, \dots, p_k$ ) { ... }  
    void P_2( $p_1, \dots, p_{k'}$ ) { ... }  
    void P_3( $p_1, \dots, p_l$ ) { ... }  
    void P_4( $p_1, \dots, p_{l'}$ ) { ... }  
    ...  
    void P_o( $p_1, \dots, p_{z'}$ ) { ... }  
}
```

Monitors

Pure Monitor. A pure monitor does not have condition variables. It only provides mutual exclusion.

```
Monitor M()
```

```
{  
    function_1(){...};  
    function_2(){...};  
    ...;  
    function_n(){...};  
}
```

```
Thread_1 ()
```

```
{  
    ...;  
    M.function_k();  
    ...;  
}
```

```
Thread_2 ()
```

```
{  
    ...;  
    M.function_k-1();  
    ...;  
}
```

```
Thread_3 ()
```

```
{  
    ...;  
    M.function_k+1();  
    ...;  
}
```


Monitors

Example of a Pure Monitor. A pure monitor does not have condition variables. It only provides mutual exclusion.

```
Monitor M()
```

```
{  
    var count=10;  
    void Inc(){count++;};  
    void Dec(){count--};  
}
```

```
Thread_1 ()
```

```
{  
    ...;  
    M.Inc();  
    ...;  
}
```

```
Thread_2 ()
```

```
{  
    ...;  
    M.Dec();  
    ...;  
}
```

Pure Monitors Vs Mutex locks

```
Monitor M()  
{  
    function_1(){...};  
    function_2(){...};  
    ...;  
    function_n(){...};  
}  
Thread ()  
{  
    ...;  
    M.function_k();  
    ...;  
}
```

```
function_1(){...};  
function_2(){...};  
    ...;  
function_n(){...};  
Mutex M = 1;  
Thread ()  
{  
    ...;  
    M.acquire();  
    function_k();  
    M.release();  
    ...;  
}
```

Condition Variables

Definition

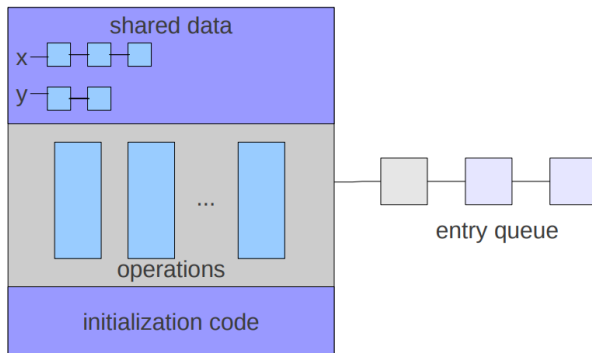
Condition Variables: A particular type of data structure that is used by monitors for process synchronization (Queue + two primitives).

- Each condition variable has a queue associated with it. The queue is made of PCBs (Process Control Blocks).
- Two operations are invocable on condition variable x :
 - $x.wait()$: The process invoking this operation is suspended (**giving back the lock**), til another process invokes $x.notify()$ to wake it up.
Giving back the lock means: release the lock so that other processes outside the monitor try to acquire it
 - $x.notify()$: Resumes exactly one suspended process from the x variable queue (**no effect if no process is blocked**).

Actually, it is more than resuming one suspended process... as we will see.

Monitors with Condition variables

Note that, a monitor with N condition variables has $N+1$ queues. Example:
A monitor with two condition variables x and y :



Condition Variables

If a process P is inside the monitor and because a condition is not satisfied yet, the process can either run `wait()` or **notify()**:

- Blocks itself and **gives back** the monitor lock [`wait()` case] .
- Wakes-up a blocked process (if there is any), then either goes to wait (blocks itself) or continues execution [`notify()` case].

Also, if a process is blocked inside a monitor, it is considered as if that process is outside the monitor.

Why two semantics? If a process Q executes `x.notify()` while another process P is currently blocked in the `x`-queue then: If P is allowed to resume, both processes will be **active** inside the monitor. [Not good]

One of the processes must wait for the other to leave the monitor or wait for another condition.

Condition Variables

Calling **wait()** primitive on a conditional variable **x** has two effects:

1. Suspends the thread which called it, and place it in x-queue.
2. Unlock monitor (the monitor's lock is given back to the entry queue).

Calling **notify()** primitive on a conditional variable **x** has one effect:

1. Wake up one waiting process in x-queue (if any).

But, which thread is allowed to continue execution in the monitor? Two notification disciplines:

1. *Notify and continue.*
2. *Notify and wait.*

Condition Variables

- 1. Notify and continue.* The notifying thread keeps the monitor lock.
 - Thread that was awakened, will wait at the entry queue (Monitor lock).
 - The condition which that thread waited for may not be true any more once that thread resumes (**Process has to double-check the condition**).
 - Used in Java Monitors.
- 2. Notify and wait.* The notifying thread is blocked and the notified one gets the lock.
 - Awakened thread executes immediately (gets the lock from notifier).
 - The condition which the thread waited for is certainly true (**no checking**).
 - The notifying process is blocked at the entry queue (goes out).

`if(Condition) wait();` **Vs** `while(Condition) wait();`

Condition variables

Consider a bakery where a cook makes bread for clients to eat. The cook makes one bread at a time and each client eats one bread at a time:

```

numBread=0;
Condition variable Q;

Cook()
{
    //Cook bread
    numBread++;
    If (NumClient>0)
    {
        Q.notify();
    }
}

Client()
{
    //Look for bread
    if(numBread == 0)
    {
        numClient++;
        Q.wait();
        numClient--;
    }
    //Grab one bread
    numBread--;
}

```

If we decide the semantic of `Notify()` to be **Notify and Continue**, then there will be errors, and the entire program goes wrong [[Try the execution scenario of Client-1; Cook; Client-2; Client-1](#)].

Condition variables

Consider a bakery where a cook makes bread for clients to eat. The cook makes one bread at a time and a client eats one at a time:

```
numBread=0;
Condition variable Q;

Cook()
{
    //Cook bread
    numBread++;
    If (NumClient>0)
    {
        Q.notify();
    }
}

Client()
{
    //Look for bread
    while(numBread == 0)
    {
        numClient++;
        Q.wait();
        numClient--;
    }
    //Grab one bread
    numBread--;
}
```

We need to transform the `If()` statement to a `While()` statement. Now, everyone can have their bread!

Monitors

Exercise. Consider a system with N threads (persons) and K ATMs. Each thread occupies one ATM for a while then leaves it:

Monitor M	Thread (Integer i)
{	{
Condition variables: Q;	M.Take();
Local variables: Nbr_ATM = K;	Use_ATM();
void Take() { ... }	M.Leave();
void Leave() { ... }	}
}	


```
void Main(Integer N){ ParBegin(Thread(1), ..., Thread(N))}
```

Do. Write code for Take() and Leave() irrespective of which Notify() semantic is being considered.

Monitors in Java

Monitors in Java

Monitors in Java

Java has monitors built in. You can turn any object into a monitor by adding the keyword **synchronized** to the declaration of one of its methods:

```
Public class Monitor_Example
{
    Public synchronized void Critical_section_1() { ... }
    Public synchronized int Critical_section_2() { ... }
    ...
    Public synchronized Object Critical_section_n() { ... }
}
```

Java uses an unnamed condition variable per monitor.

There exist three primitives for a CV: `wait()`, `notify()`, and `notifyAll()`.

No name for condition variable means everyone should wait in the same queue (e.g., readers and writers).

Monitors

Java Monitor. Java monitors have one default condition variable on which you can call two special purpose primitives.

```
public class M{
{
    synchronized void method_1(){...};
    synchronized void method_2(){...};
    ...;
    synchronized void method_n(){...};
}
Thread_1 ()      Thread_2 ()      Thread_3 ()
{                {                {
    ...;          ...;          ...;
    M.method_k(); M.method_k-1(); M.method_k+1();
    ...;          ...;          ...;
}                }                }
```

Monitors in Java

Example. Consider a class defining an object Counter:

```
class Counter                                Counter counter;  
{                                              Thread()  
    private int count = 0;                    {  
    public void Increment()                  counter.Increment();  
    {                                        }  
        count++;  
    }  
}
```

If more than two Threads are executing concurrently, the value of count will be incremented under race condition (**subject to data corruption**).

I.e., after one execution of 100 threads, we may get $\text{count} < 100$.

Monitors in Java

Example. Consider a class defining an object Counter:

```
class Counter
{
    private int count = 0;
    public void synchronized Increment();
    {
        count++;
    }
}
```

Now for N threads and for one execution each, we will get $\text{count} = N$.

I.e., after one execution of 100 threads, we'll certainly get $\text{count} = 100$.

Monitors in Java

In Java, the unnamed condition variable is used as follows:

- A thread can block itself on the condition variable:

using `wait()` [Monitor lock is released]

- A thread can wake up another thread blocked on the condition variable:

using `notify()` [notify & continue]

- A thread can wake up all threads blocked on the condition variable:

using `notifyAll()` [notify & continue]

Java Monitors use two queues:

- 1 **Entry queue.** A queue used to access the monitor.
- 2 **Unnamed queue.** A queue used inside the monitor.

Both queues are not FIFO. Which waiting process goes next is a matter of luck and timing (i.e., random).

Monitors in Java

Note: **Java monitors are not like traditional critical sections.**

Declaring a method synchronized does not imply that only one thread may execute that method at a time, as would be the case with critical section. in fact, declaring a method synchronized implies that only one thread may invoke that method on a **particular object** at any given time.

e.g., For two instances of an object, say A and B, the method M1 can be invoked by two threads at the same time but on different object: A.M1()
and B.M1()

Monitors in Java

The semantics of `wait()`, `notify()`, and `notifyAll()` in Java:

- **`wait()`**: Wait in the Unnamed queue & release the monitor lock.
- **`notify()`**: Wake up one blocked process, and move it from the CV queue to the entry queue.
- **`notifyAll()`**: Wake up every blocked process and move all of them from the CV queue to the entry queue. Each one of them will check whether it is the right time for it to really wake up once it gets the lock to enter the monitor.

Monitors in Java: Notify Vs NotifyAll

When the concerned queue is used by only one class of threads or by two different classes of threads such that the number of threads in each class is **1**, then `Notify()` can be used.

When the concerned queue is used by two different classes of thread and that the number of threads in each class is ≥ 2 , then `NotifyAll()` **must** be used. Else Deadlock **may** occur.

E.g., Scenario where a wrong thread is waken up and the notify signal is lost.

End of Process synchronization

Central Processing Unit Scheduling

Recall

Single-tasking OS. In this type of systems, only one process is resident in the main memory at a given time:

- Only one process can be executed at a time.
- When a process performs an I/O operation, the CPU sits idle.
- The system is less productive and wastes lots of CPU time.

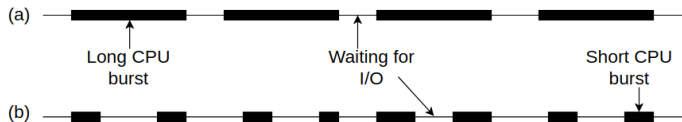
Multitasking OS. In this type of systems, multiple processes are resident in the main memory at a given time (multiprogramming):

- Programs execute either in fake or real concurrently ($\#$ CPU).
- When a process performs an I/O operation, another process uses CPU.
- The system is more productive and waiting time is used for useful work.

Importance of process scheduling

Process behavior

Nearly all processes alternate burst of computing with disk I/O requests:



Two types of processes (*Key factor is the length of CPU burst*):

Compute-bound. Processes spend most of their time computing (e.g., a).

have long CPU burst and infrequent I/O waits

I/O-bound. Processes spend most of their time waiting for I/O (e.g., b).

have short CPU burst and frequent I/O waits (they do not compute much between I/O)

CPU Scheduler

Definition

Scheduler: A program that, based on a scheduling algo, decides which process is allocated the CPU, admitted to the ready queue, or swapped in to RAM or swapped out to the HDD/SSD .

- It acts to switch process states and hence provides multiprocessing.
- Some schedulers can be preemptive, non preemptive, or hybrid.
- Main objectives:
 - Make CPU utilization higher (Increase the throughput).
 - Increase the degree of multiprogramming.
 - Guarantee fairness.
 - Decrease the turnaround time, waiting time, and response time.

Precisely. The CPU scheduler that is responsible for selecting a process for execution (allocated CPU) is called: short-term scheduler.

CPU Scheduler

The operating system uses a queue named **ready queue** to line up processes that are waiting to get allocated the CPU.

The elements of the queue are basically PCBs of the processes.

The way how a process is selected to be allocated the CPU depends on which **scheduling algorithm** is adopted.

There exists two main classes of short-term scheduling algorithms:

❶ **Preemptive:** Is the class of scheduling algorithms that can interrupt (or moves) a process during its execution because:

- The current process requested an I/O operation ($R_u \rightarrow B_I$).
- The current process had to wait for some events ($R_u \rightarrow B_I$).
- A context switching occurred following a timer interrupt ($R_u \rightarrow R_e$).
- An I/O Completion, process brought back to ready-Q ($B_I \rightarrow R_e$).
- The current process terminates ($R_u \rightarrow T_r$).

CPU Scheduler

The operating system uses a queue named **ready queue** to line up processes that are waiting to get allocated the CPU.

The elements of the queue are basically PCBs of the processes.

The way how a process is selected to be allocated the CPU depends on which **scheduling algorithm** is adopted.

There exists two main classes of scheduling algorithms:

② **Non-preemptive:** Is the class of scheduling algorithms that can interrupt (or moves) a process only when:

- The current process requested an I/O operation ($R_u \rightarrow B_I$).
- The current process had to wait for some event, e.g., mutex ($R_u \rightarrow B_I$).
- The current process terminates ($R_u \rightarrow T_r$).

Non-preemptive is still used on certain hardware platforms (e.g., resource constraints devices — alarm systems, sensors, IoT devices, ...).

Types of Scheduler

There are three types of Schedulers in a system:

- **Long-term scheduler.** A.k.a., Job scheduler. It decides which jobs or processes are to be admitted to the ready queue. It may admit or delay the process based on some parameters. It decides the degree of multiprogramming, it is less frequent, and it is present on certain OSs.
- **Medium-term scheduler.** Part of the swapper. It decides which process to swap out from the memory to disk (has not been active for some time, has a low priority, or a process which is page faulting frequently) and which ones to swap in to the memory.
- **Short-term scheduler.** A.k.a., CPU scheduler. It decides which process in the ready queue gets the CPU for execution.

Scheduling criteria

The short-term scheduler applies a scheduling algorithm to select a process among many to be allocated the CPU by the dispatcher. The most known algorithms are:

- FCFS (First Come, First Served).
- Priority based scheduling algorithm.
- Multilevel queue scheduling.
- RR (Round Robin).
- SJF (Short Job First).
- ...

At the same time, there exists many metrics that can be used to compare different scheduling algorithm (This depends on the type of the system):

- **CPU utilization**: In real systems, it should range from 40% to 90%.
- **Throughput**: Processes per times unit.
- **Turnaround time**: Time from process submission to completion.
- **Waiting time**: Total amount of time spent in the ready queue.
- **Response time**: From submission of a request to first response.

FCFS (First Come, First Served) Scheduling Algorithm

It consists of selecting the first process that arrived at the ready queue:

- It adopts a FIFO (First In First Out) queue.
- No preemption is allowed (non preemptive algorithm).
- No priority is used (system processes are penalized).
- Blocked processes which becomes ready join the back of the queue.
- Short processes have to suffer a long waiting time because of processes with longer burst time arrived first (**Convoy effect**).
- The response time is bad for short processes which arrived late.

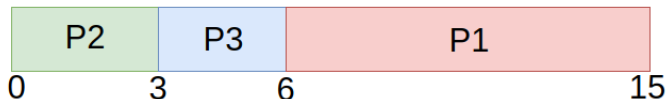
FCFS (First Come, First Served) Scheduling Algorithm

Example. Consider 3 processes P1, P2, & P3, with burst time: 9, 3, 3 (Time Units) respectively. If the arrival order is P1, P2, P3 at $t=0$, the FCFS scheduling gives the following process execution Gantt chart:



The average waiting time $\overline{t_{wait}} = \frac{(0+9+12)}{3} = 7 \text{ TUs}$.

However, If we switch the order of arrival:



The average waiting time is much better $\overline{t_{wait}} = \frac{(0+3+6)}{3} = 3 \text{ TUs}$.

SJF (Shortest Job First) Scheduling Algorithm

It consists in allocating the CPU to the process that has the shortest burst time (assumes run times are known in advance).

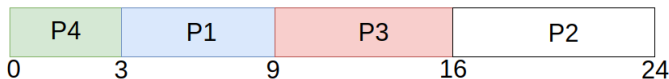
- It can be non preemption or preemptive (SRTF: Shortest Remained Time First).
- Very important process (system process) which burst is long may not be executed (starvation).
- The response time is bad for process with longer burst time.
- FCFS is used in case of processes with same burst time.

It cannot be implemented as the short-term scheduling. There is no way to know the CPU burst time of a processes (maybe an $\approx$).

Can be used in systems where similar tasks are done everyday.

SJF (Shortest Job First) Scheduling Algorithm

Example. Consider 4 processes P1, P2, P3, & P4, with burst time: 6, 8, 7, 3 (Time Units) respectively. The SJF scheduling gives the following process execution Gantt chart:

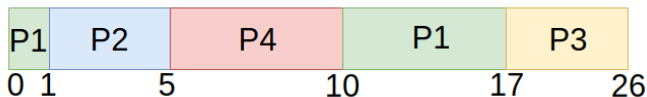


The average waiting time $\overline{t_{wait}} = \frac{(0+3+9+16)}{4} = 7 \text{ TUs}$.

By placing the short processes before the long ones, the waiting time decreases.

SJF (Shortest Job First) Scheduling Algorithm

Example (SRTF). Consider 4 processes P1, P2, P3, & P4, with burst time: 8, 4, 9, 5 (Time Units) respectively, arriving at 0, 1, 2, 3 respectively. The SRTF scheduling gives the following process execution Gantt chart:



The average waiting time $\overline{t_{wait}} = \frac{(9+0+15+2)}{4} = 6.5 \text{ TUs}$.

This scheduling algorithm is also called optimal algorithm.

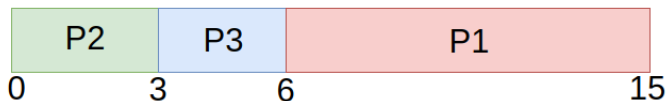
Priority-based Scheduling Algorithm

It consists in allocating the CPU to the process that has the highest priority.

- Each process has a priority expressed by an integer (e.g., 0 is the highest priority).
- Processes with higher priority and longer burst time will penalize the other ones.
- Processes with lower priority are subject to starvation.
- FCFS is used when processes have the same priority.

Priority-based Scheduling Algorithm

Example. Consider 3 processes P1, P2, & P3, with burst time: 9, 3, 3 (Time Units) respectively and priority 2, 0, 1 respectively. The Priority scheduling gives the following process execution Gantt chart:



The average waiting time $\overline{t_{wait}} = \frac{(0+3+6)}{3} = 3 \text{ TUs}$.

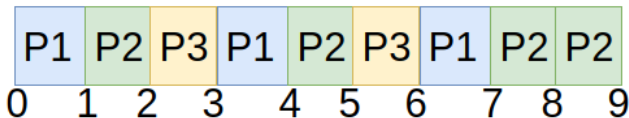
RR (Round Robin) Scheduling Algorithm

It consists in alternatively allocating the CPU to each process for a certain amount of time called **quantum** or **time slice**.

- It is a preemptive algorithm.
- Used in interactive systems.
- No priority. All processes are considered equal.
- The CPU is at most used for a fixed amount of time called quantum by each process (**because: I/O operation or short burst time**).
- When the quantum is too small, there will be lot of context switching.
- When the quantum is too big, the RR becomes FCFS.

RR (Round Robin) Scheduling Algorithm

Example. Consider 3 processes P1, P2, & P3, with burst time: 3, 4, 2 (Time Units) respectively. The RR scheduling with $Q=1TU$ gives the following process execution Gantt chart:



The average waiting time $\overline{t_{wait}} = \frac{(4+5+4)}{3} = 4.33 \text{ } TUs.$

MLQ (MultiLevel Queue) Scheduling Algorithm

It consists of having multiple queues for each class of process, each applying a particular scheduling algorithm.

- Process are classified and the queues are prioritized.
- Each process is redirected to its class waiting queue.
- The CPU is allocated to a process of a particular queue depending on the queue priority.

Another variant of MLQ, called **Multilevel Feedback Queue algorithm**, processes move from one queue to another (to prevent starvation).

e.g., Round Robin where each time (round) the process is escalated to a higher priority queue.

Note that there are many other scheduling algorithms: priority RR, preemptive priority, lottery scheduling, ... and more.

End of CPU scheduling

- End.